



UNIVERSIDADE DO ESTADO DO PARÁ
CAMPUS XXIII – PARAUAPEBAS

ATIVIDADE DE
ESTRUTURA DE DADOS PESQUISA E ORDENAÇÃO

ALUNO: FABIO RIBEIRO OLIVEIRA FILHO

PARAUAPEBAS - PA

2026

- ① R: A principal diferença entre lista, fila e pilha está na forma como os elementos são organizados e acessados. A lista permite inserir e remover elementos em qualquer posição, sendo mais flexível. A fila segue a lógica FIFO (primeiro a entrar, primeiro a sair), ou seja, o primeiro elemento inserido será o primeiro removido. Já a pilha segue a lógica LIFO (último a entrar, primeiro a sair), onde o último elemento inserido é o primeiro a ser removido.
- ② R: Um exemplo é a fila do supermercado (ex: Mix Market). As pessoas entram no final da fila e são atendidas por ordem de chegada. Quem chega primeiro é atendido primeiro.
- ③ R: Um exemplo é uma pilha de pratos. O último prato colocado é o primeiro a ser retirado.
- ④ R: A lista é ideal para organizar tarefas de um estudante. Permite adicionar, remover ou mudar itens em qualquer posição. Diferente da fila e da pilha, é mais flexível.
- ⑤ R: Tipos de dados:
Identificador: inteiro Valor estimado: float
Ponto de partida: string Tempo estimado: inteiro
Destino: string
Estrutura escolhida: Lista duplamente encadeada
Permite percorrer para frente e para trás. Facilita inserção e remoção no meio da lista. É mais flexível para reorganização de rotas e melhor para consultas frequentes.

04.05.26

D S T Q Q S S

6. R=

ID	Partida	Destino	Valor estimado	Tempo Utilizado
1	Cidade Nova	Rio Verde	R\$ 5,00	15 min
2	União	Shopping	R\$ 10,00	25 min
3	Palmares	Centro	R\$ 15,00	40 min
4	Rio Verde	IFPA	R\$ 5,00	20 min
5	Cidade Jardim	Hospital	R\$ 10,00	25 min

7. R= Pseudo código

Início

para cada rota na lista
se rota.valor ≤ 12 então
mostrar rota

fim se

fim para

mostrar "Limite utilizado: 12"

Fim

Código em C

```
#include <stdio.h>
```

```
struct Rota {
```

```
    int id;
```

```
    char partida[50];
```

```
    char destino[50];
```

```
    float valor;
```

```
    int tempo;
```

```
};
```

```
int main() {
```

```
    struct Rota rotas[5] = {
```

```
        {1, "Cidade Nova", "Rio Verde", 5, 15},
```

```
        {2, "União", "Shopping", 10, 25},
```

```
        {3, "Palmares", "Centro", 15, 40},
```

```
        {4, "Rio Verde", "IFPA", 5, 20},
```

```
        {5, "Cidade Jardim", "Hospital", 10, 25},
```

```
    };
```



```

7. R int i;
    float limite = 12;
    for (i = 0; i < 5; i++) {
        if (notas[i].valor <= limite) {
            printf("Rota %d = %s -> %s \n", notat[i].id, notas[i].partida, notas[i].destino);
        }
    }
    printf("Limite utilizado: %.2f \n", Limite);
    return 0;
}

```

3. **R** = Pseudocódigo

Início

para i de 0 até 4

para j de i+1 até 5

se tempo[i] > tempo[j]

trocar posições

fimse

limpara

limpara

Fim

Código em C

```
int i, j;
```

```
struct Rota temp;
```

```
for (i = 0; i < 5; i++) {
```

```
    for (j = i + 1; j < 5; j++) {
```

```
        if (notas[i].tempo > notas[j].tempo) {
```

```
            temp = notas[i];
```

```
            notas[i] = notas[j];
```

```
            notas[j] = temp;
```

```
        }
```


8. R= Ajuda o estudante a escolher rotas mais rápidas. Facilita chegar no horário nas aulas e melhora o planejamento de dia.

9. R= Tipo 1: simples
Tipo 2: intermediária
Tipo 3: prioritária

Pos.	Nome	CPF	Tipo	Hora
1	João	123456789-10	1	8h
2	Maria	214579653-15	2	8:05h
3	Pedro	987654357-23	1	8:10h
4	Ana	654321032-98	3	8:15h
5	Lucas	765431234-37	2	8:20h
6	Carla	987656437-45	1	8:25h

10. R= Pseudocódigo

Início

para cada pessoa na fila

se tipo == 1

inserir na fila A

senão

inserir na fila B

fim se

fim para

mostrar fila A

mostrar fila B

Fim

10. R=

```
int main () {
    struct Pessoa fila[6] = {
        { "João", 1 }, { "Mônica", 2 }, { "Pedro", 1 },
        { "Ana", 3 }, { "Lucas", 2 }, { "Carla", 1 }
    };
```

```
    struct Pessoa filaA[6]; filaB[6];
    int i, a=0, b=0;
```

```
    for (i=0; i<6; i++) {
        if (fila[i].tipo == 1) {
            // else {
            filaB[b++] = fila[i];
        }
    }
```

```
    printf("Fila A: \n");
```

```
    for (i=0; i<a; i++) {
        printf("%s \n", filaA[i].nome);
    }
```

```
    printf("Fila B: \n");
```

```
    for (i=0; i<b; i++) {
        printf("%s \n", filaB[i].nome);
    }
```

```
    return 0;
```

11. R= Dividir a fila melhora a organização e permite atender casos ~~mais~~ simples mais rápido. Por outro lado, pode gerar desigualdade no tempo de espera para atendimentos mais complexos. Esse modelo é inútil em hospitais e serviços públicos em Portugal.

12. R=

Ocorrência	Pesquisa	Data	Consequência
1	Entrar na Faculdade	2024	Formação
2	Curso Técnico	2023	Fortalecimento
3	Aprender Programação	2025	Emprego
4	Fazer Estágio	2026	Experiência

13. R= Pseudocódigo

Início

Enquanto pilha não vazia
remover topo
se topo == decisão-chave
porar

fimse
fimenquanto
mostrar pilha restante

Fim

Código em C

```
int main() {
```

```
    char pilha[4][50] = {
```

```
        "Faculdade", "Curso Técnico",  
        "Programação", "Estágio"
```

```
    };
```

```
    int topo = 3;
```

```
    while (topo > 0) {
```

```
        if (strcmp(pilha[topo], "Programação") == 0) {  
            break;
```

```
        }
```

```
        topo--;
```

```
    }
```

```
    printf("Restante: \n");
```

```
    for (int i = 0; i <= topo; i++) {
```

```
        printf("%s \n", pilha[i]);
```

```
    }
```

```
    return 0;
```


13. R= A pilha é adequada para esse tipo de processo porque trabalha com o princípio do último a entrar primeiro a sair. Isso faz com que as decisões mais recentes sejam analisadas primeiro, o que é útil quando se busca uma decisão específica mais atual. Além disso, a pilha permite interromper o processo assim que a decisão-chave é encontrada, tornando a execução mais eficiente e organizada.

14. R=

Ordem	Tela
1	Tela inicial
2	Notas
3	Matrícula
4	Histórico
5	Mensagens

Pseudocódigo

Início

enquanto topo != tela inicial
desempilhar
fimenquanto
mostrar topo

Fim

Código em C #include <stack.h>

#include <string.h>

int main() {

char pilhas[5][50] = {

"Tela inicial", "Notas",

"Matrícula", "Histórico", "Mensagens"

};

int topo = 4;

while (strcmp(pilhas[topo], "Tela inicial") != 0) {

topo--;

}

14. $R = \text{ponto} ("Tela atual: %s \backslash n", \text{pilha}[\text{topo}]);$
 $\text{return } 0;$

15. $R =$
~~(Listas, filas e pilhas são estruturas)~~

Listas, filas e pilhas são estruturas de dados lineares, mas possuem comportamentos diferentes. A lista permite inserir e remover elementos em qualquer posição, sendo bastante flexível. Já a fila segue o princípio FIFO, onde o primeiro elemento a entrar é o primeiro a sair, como em filas de atendimento. A pilha segue o modelo em que o último elemento inserido é o primeiro a ser removido.

Na prática, a lista pode ser usada para organizar tarefas de um estudante, a fila em atendimentos em hospitais de triagem e a pilha em históricos de navegação. Como limitação, a lista pode ser menos eficiente em grandes volumes de dados, a fila não permite acesso direto a elementos do meio e a pilha também possui acesso restrito apenas ao topo.

Em Porquapebas, essas estruturas podem ser aplicadas em filas de postos de saúde, organização de notas ou controle de acesso em sistemas. Entre elas, a lista parece mais intuitiva neste momento, pois permite maior flexibilidade e é mais fácil de entender e aplicar no dia a dia.